

Lab 3: Cache Simulator — Analysis Report

Sharique - 28369

February 12, 2026

Contents

1	Task 1: Trace File Generation	2
2	Task 2: Baseline L1 Cache Analysis	2
3	Task 3: Varying Associativity (2, 4, 8-way)	2
4	Task 4: Varying Block Size (16, 32, 128 bytes)	3
5	Task 5: Varying Replacement Policy	4
6	Task 6: Write-Through Policy	4
7	Task 7: Prefetching	5
8	Task 8: Adding L2 Cache	5
9	Task 9: MESI Coherence Protocol Assessment	6
10	Summary of Key Findings	7

1 Task 1: Trace File Generation

Ten trace files were generated, each with 500,000 memory accesses, using the `trace_generator` tool. The default write ratio is 30% (70% reads, 30% writes) unless overridden.

Table 1: Generated Trace Files

#	Trace Name	Pattern	Key Parameter
1	sequential_500k	Sequential	stride=64
2	random_500k	Random	—
3	strided64_500k	Strided	stride=128
4	looping_2x4_500k	Looping	2 loops, 4 reps
5	mixed_high_loc_500k	Mixed	locality=0.8
6	mixed_low_loc_500k	Mixed	locality=0.2
7	write_intensive_500k	Mixed	write ratio=0.8
8	low_write_500k	Mixed	write ratio=0.2
9	seq_4proc_500k	Sequential	4 processors
10	rand_4proc_500k	Random	4 processors

2 Task 2: Baseline L1 Cache Analysis

Configuration: L1 = 32KB, 1-way (direct-mapped), 64-byte blocks, LRU, Write-Back.

Table 2: Task 2 — Baseline Results (1-way, 64B blocks, LRU, Write-Back)

Trace	Reads	Writes	Hits	Misses	WB	Hit%	Comp.	Cap.	Conf.	Coh.
Sequential	349,820	150,180	0	500,000	150,034	0.00	512	0	499,488	0
Random	349,240	150,760	15,715	484,285	149,119	3.14	512	0	483,773	0
Strided-64	350,614	149,386	0	500,000	149,306	0.00	256	0	499,744	0
Looping	350,178	149,822	499,998	2	0	100.00	2	0	0	0
Mixed Hi-Loc	349,812	150,188	121,793	378,207	117,146	24.36	512	0	377,695	0
Mixed Lo-Loc	349,466	150,534	120,988	379,012	117,655	24.20	512	0	378,500	0
Write-Heavy	100,136	399,864	119,598	380,402	305,574	23.92	512	0	379,890	0
Read-Heavy	400,142	99,858	120,738	379,262	79,880	24.15	512	0	378,750	0
Seq 4-Proc	349,660	150,340	0	500,000	150,179	0.00	512	0	499,488	0
Rand 4-Proc	350,505	149,495	15,659	484,341	147,898	3.13	512	0	483,829	0

Observation: The direct-mapped cache performs terribly for most workloads. Sequential and strided patterns yield 0% hit rate because every access maps to a new block and the working set far exceeds the 32KB cache. The looping pattern achieves a perfect 100% hit rate since only 2 unique addresses fit trivially in cache. Mixed patterns hover around 24% due to partial spatial reuse. Over 99.9% of all misses are **conflict misses**, highlighting the fundamental limitation of direct-mapped caches. Write ratio doesn't affect hit rate but significantly impacts write-backs (305K for 80% writes vs. 80K for 20% writes).

3 Task 3: Varying Associativity (2, 4, 8-way)

Configuration: Same as Task 2 but associativity changed to 2, 4, and 8.

Table 3: Task 3 — Hit Ratio (%) by Associativity

Trace	1-way	2-way	4-way	8-way
Sequential	0.00	0.00	0.00	0.00
Random	3.14	3.15	3.15	3.15
Strided-64	0.00	0.00	0.00	0.00
Looping	100.00	100.00	100.00	100.00
Mixed Hi-Loc	24.36	24.59	25.38	25.39
Mixed Lo-Loc	24.20	24.42	25.32	25.32
Write-Heavy	23.92	24.09	25.28	25.30
Read-Heavy	24.15	24.35	25.31	25.33
Seq 4-Proc	0.00	0.00	0.00	0.00
Rand 4-Proc	3.13	3.14	3.14	3.12

Observation: Increasing associativity provides **marginal improvement** ($\sim 1\%$) for mixed patterns by reducing conflict misses. The sequential and strided patterns see no benefit because their working sets far exceed the cache size—the misses are fundamentally due to the data not fitting, not due to conflicts within a set. The looping pattern remains perfect. The biggest gain is from 1-way to 4-way for mixed patterns ($\sim 24\% \rightarrow 25.3\%$). Beyond 4-way, diminishing returns are evident (4-way vs. 8-way shows $<0.1\%$ difference).

4 Task 4: Varying Block Size (16, 32, 128 bytes)

Configuration: Same as Task 2 but block size changed to 16, 32, and 128.

Table 4: Task 4 — Hit Ratio (%) by Block Size

Trace	16B	32B	64B (base)	128B
Sequential	0.00	0.00	0.00	50.00
Random	3.14	3.14	3.14	3.16
Strided-64	0.00	0.00	0.00	0.00
Looping	100.00	100.00	100.00	100.00
Mixed Hi-Loc	24.36	24.36	24.36	27.21
Mixed Lo-Loc	24.20	24.20	24.20	27.38
Write-Heavy	23.92	23.92	23.92	27.11
Read-Heavy	24.15	24.15	24.15	27.17
Seq 4-Proc	0.00	0.00	0.00	50.00
Rand 4-Proc	3.13	3.13	3.13	3.12

Observation: Block size has a dramatic effect on sequential access. With 128B blocks, the sequential trace jumps from 0% to **50%** hit rate because each 128B block now covers two consecutive 64-byte-stride accesses—the second access in every pair is a hit. Mixed patterns gain $\sim 3\%$ from the larger block size. However, random and strided-64 patterns see negligible change because larger blocks don’t help when access patterns don’t exhibit spatial locality within the block. Notably, 16B and 32B blocks give **identical results** to 64B for this particular trace set, suggesting the trace generator’s stride perfectly aligned with block boundaries.

5 Task 5: Varying Replacement Policy

Configuration: 32KB, 4-way, 64B blocks, Write-Back (4-way used instead of 1-way because replacement policy has no effect on direct-mapped caches—there’s only one slot per set).

Table 5: Task 5 — Hit Ratio (%) by Replacement Policy (4-way)

Trace	LRU	FIFO	PLRU	Random
Sequential	0.00	0.00	0.00	0.00
Random	3.14	3.13	3.10	3.14
Strided-64	0.00	0.00	0.00	0.00
Looping	100.00	100.00	100.00	100.00
Mixed Hi-Loc	25.38	24.36	21.62	24.36
Mixed Lo-Loc	25.30	24.18	21.24	24.21
Write-Heavy	25.28	23.90	20.31	23.90
Read-Heavy	25.31	24.12	21.02	24.13
Seq 4-Proc	0.00	0.00	0.00	0.00
Rand 4-Proc	3.14	3.14	3.12	3.13

Observation: For workloads with some locality (mixed patterns), the ranking is: **LRU** > **FIFO** $\approx$ **Random** > **PLRU**. LRU outperforms all others by $\sim 1\text{--}4\%$ because it makes the best eviction decisions based on recency. Surprisingly, PLRU performs worst ($\sim 21\%$) because its tree-based approximation makes poor eviction choices under this access pattern. FIFO and Random perform similarly ($\sim 24\%$). For streaming patterns (sequential, strided), all policies are equally ineffective since every access is a miss regardless of eviction strategy.

6 Task 6: Write-Through Policy

Configuration: Same as Task 2 but using Write-Through instead of Write-Back.

Table 6: Task 6 — Write-Through vs. Write-Back Comparison

Trace	Hit%	WB (Write-Back)	WB (Write-Through)
Sequential	0.00	150,034	150,076
Random	3.14	149,119	145,930
Strided-64	0.00	149,306	149,306
Looping	100.00	0	0
Mixed Hi-Loc	24.36	117,146	113,635
Mixed Lo-Loc	24.20	117,655	113,967
Write-Heavy	23.92	305,574	303,976
Read-Heavy	24.15	79,880	75,798

Observation: The hit ratio is **identical** between Write-Back and Write-Through because the write policy doesn’t affect whether data is found in cache. The difference is

in **memory traffic**: Write-Through sends every write directly to memory (more traffic but simpler consistency), while Write-Back defers writes until eviction (fewer memory writes but requires dirty-bit tracking). The write-back counts are slightly lower for Write-Through because dirty blocks don't accumulate—data is always clean, so evictions don't require write-backs. However, Write-Through still generates write-backs for blocks that were brought in on a read miss and later written.

7 Task 7: Prefetching

Configuration: Same as Task 2 with prefetching enabled (distance=4).

Table 7: Task 7 — With Prefetching vs. Baseline

Trace	Hit% (base)	Hit% (pref)	Misses (base)	Misses (pref)
Sequential	0.00	41.17	500,000	294,172
Random	3.14	3.14	484,285	484,285
Strided-64	0.00	41.24	500,000	293,820
Looping	100.00	100.00	2	2
Mixed Hi-Loc	24.36	24.39	378,207	378,055
Mixed Lo-Loc	24.20	24.23	379,012	378,875
Write-Heavy	23.92	23.93	380,402	380,351
Read-Heavy	24.15	24.18	379,262	379,093
Seq 4-Proc	0.00	41.14	500,000	294,278
Rand 4-Proc	3.13	3.13	484,341	484,341

Observation: Prefetching has a **massive impact on sequential/strided patterns**, improving hit rate from 0% to ~41% and reducing misses by ~41%. The prefetcher correctly predicts the next blocks in the stride pattern and fetches them before they're needed. For **random access**, prefetching has **zero effect**—the prefetched blocks are never used because the next access goes to an unpredictable location. Mixed patterns see negligible improvement (<0.1%) because their access patterns are not consistently predictable enough for the stride prefetcher to help.

8 Task 8: Adding L2 Cache

Configuration: L1 same as Task 2. L2 = 256KB, 16-way, 64B blocks, PLRU, Write-Back.

Table 8: Task 8 — L1 and L2 Statistics

Trace	L1 Hit%	L1 Misses	L2 Hit%	L2 Miss%
Sequential	0.00	500,000	43.57	56.43
Random	3.14	484,285	61.92	38.08
Looping	100.00	2	50.00	50.00
Mixed Hi-Loc	24.36	378,207	50.71	49.29

Observation: The L2 cache successfully catches a significant portion of L1 misses. For **random access**, the L2 hit rate is 61.92%—this is because the random trace’s working set ($16,320 \text{ unique addresses} \times 64\text{B} = \sim 1\text{MB}$) partially fits in the 256KB L2 with 16-way associativity reducing conflicts. For **sequential**, the L2 captures 43.57% of L1 misses. The L1 hit rates remain unchanged (the L2 doesn’t affect L1 behavior). The key takeaway: multi-level caches reduce the **effective miss penalty** by catching misses at L2 before they go to slow main memory.

9 Task 9: MESI Coherence Protocol Assessment

Configuration: Baseline L1 with multiprocessor traces (4 processors).

Table 9: Task 9 — MESI State Transitions (Sequential 4-Processor Trace)

From \ To	Modified	Exclusive	Shared	Invalid
Modified	0	104,982	0	0
Exclusive	105,000	0	0	0
Shared	0	0	0	0
Invalid	143	369	0	0

Table 10: Task 9 — MESI State Transitions (Random 4-Processor Trace)

From \ To	Modified	Exclusive	Shared	Invalid
Modified	0	103,612	0	0
Exclusive	103,638	0	0	0
Shared	0	0	0	0
Invalid	143	369	0	0

Assessment of MESI correctness:

The observed transitions are consistent with MESI protocol behavior for a single-cache simulation:

- **I → E** (369 transitions): When a processor first reads a block, it enters the *Exclusive* state since no other cache holds it.
- **I → M** (143 transitions): When a processor first writes to a block not in any cache, it goes directly to *Modified*.
- **E → M** ($\sim 105\text{K}$ transitions): When a processor writes to an *Exclusive* block, it silently upgrades to *Modified* without bus traffic (this is the key advantage of MESI over MSI).
- **M → E** ($\sim 105\text{K}$ transitions): When a modified block is written back to memory (on eviction), it transitions back to *Exclusive* on the next load.
- **No Shared state:** Since the simulator runs a single shared cache for all processors (not per-processor caches), there are no $S \rightarrow I$ invalidations or $M \rightarrow S$ downgrades. True multiprocessor coherence traffic would appear with per-processor cache simulation.

10 Summary of Key Findings

1. **Direct-mapped caches are extremely conflict-prone.** Over 99.9% of misses across all non-looping patterns are conflict misses.
2. **Associativity helps, but with diminishing returns.** The biggest gain is 1-way $\rightarrow$ 4-way ($\sim 1\%$ improvement for mixed patterns). Beyond 4-way, improvements are negligible.
3. **Block size dramatically affects spatial-locality workloads.** 128B blocks improve sequential hit rate from 0% to 50%.
4. **LRU is the best replacement policy**, outperforming FIFO, Random, and PLRU by 1–4% on mixed workloads.
5. **Write policy doesn't affect hit rate** but affects memory traffic. Write-Back is preferred for reducing bandwidth.
6. **Prefetching is highly effective for predictable patterns** (41% improvement for sequential) but useless for random access.
7. **L2 cache catches 44–62% of L1 misses**, significantly reducing the effective miss penalty.
8. **MESI transitions are correct:** $I \rightarrow E$ for first reads, $E \rightarrow M$ for writes (silent upgrade), $M \rightarrow E$ on write-back.